

OKF bundle은 “DBMS를 이해하기 위한 semantic/schema catalog”**로 보면 됩니다. Text-to-SQL agent에서는 OKF를 SQL 생성 전 단계의 schema linking + semantic grounding + join rule lookup에 사용해야 합니다.

bundle = directory tree of markdown concepts

```
llm-wiki
├── Bundle List
└── Concept File
```

OKF spec에서는 bundle이 “self-contained hierarchical collection”이고, concept ID는 bundle 내부 파일 경로에서 `.md`를 제거한 값입니다. 예를 들어 `tables/users.md`는 `tables/users` concept ID가 됩니다.

그래서 실제 ID는 다음처럼 **bundle namespace**를 붙여야 충돌이 없습니다. LLM wiki 레벨에서는 `bundle_id + concept_id`를 전역 ID로 삼는다.

```
ga4:tables/events
stackoverflow:tables/posts
crypto_bitcoin:tables/transactions
```

2. 하나의 LLM wiki 안에 여러 bundle이 있을 때 검색 구조

질의는 바로 RAG로 문서를 던지기보다, 먼저 **schema discovery** 단계를 거치는 것이 좋습니다.

```
User Query
  ↓
Intent Router
  ↓
Bundle Discovery
  ↓
Concept Discovery
  ↓
Schema / Join / Metric 확인
  ↓
SQL 또는 tool call 생성
  ↓
Answer
```

예를 들어 사용자가 “VIP 고객은 몇 명이야?”라고 물으면:

1. 질의 키워드 추출
 - VIP
 - customer
 - grade
 - count
2. bundle 후보 찾기
 - ecommerce
 - crm
 - ga4는 보조 후보
3. concept 후보 찾기
 - tables/customers
 - metrics/vip_customer_count
 - glossary/vip
4. schema section 읽기
 - customers.grade
 - customers.id

```
- customers.name
```

5. SQL 생성

3. 각 bundle에 반드시 넣어야 할 discovery용 파일

OKF의 `index.md` 는 선택 사항이지만, 여러 bundle을 LLM wiki로 운영하려면 사실상 필수로 두는 것이 좋습니다. OKF README도 `index.md` , directory hierarchy, YAML frontmatter 기반 구성을 강조합니다. ([GitHub](#))

추천 구조:

```
ga4/
  index.md
  bundle.yaml          # 비표준이지만 운영용 manifest
  glossary/
    user.md
    session.md
  datasets/
    analytics.md
  tables/
    events.md
    users.md
  metrics/
    active_users.md
    purchase_revenue.md
  joins/
    user_event_join.md
```

`index.md` 에는 사람이 읽는 설명을, `bundle.yaml` 에는 agent가 빠르게 읽는 manifest를 둡니다.

```
bundle_id: ga4
title: GA4 E-commerce
domain: analytics
concept_types:
  - dataset
  - table
  - metric
  - dimension
  - join
primary_tables:
  - tables/events
  - tables/users
keywords:
  - ga4
  - ecommerce
  - event
  - user
```

4. concept/schema를 찾기 위한 frontmatter 설계

각 concept markdown 상단에 discovery용 metadata를 강화합니다.

```
---
type: table
title: Customers
bundle_id: ecommerce
concept_id: tables/customers
```

```
resource: postgres://shop/customers
tags: [customer, vip, crm]
aliases: [users, members, clients]
columns: [id, name, email, grade, point_balance]
primary_key: id
---
```

Schema

```
| column | type | meaning |
|---|---|---|
| id | int | customer id |
| grade | string | NORMAL, VIP |
| point_balance | int | current points |
```

Query Examples

```
VIP customers:
SELECT * FROM customers WHERE grade = 'VIP'
```

중요한 것은 **frontmatter**는 검색용 카드, **body**는 상세 설명으로 쓰는 것입니다.

5. LLM wiki의 schema discovery index

여러 bundle을 운영한다면 ingestion 시 다음 인덱스를 만들어야 합니다.

```
bundle_index
- bundle_id
- title
- domain
- tags
- path

concept_index
- global_id
- bundle_id
- concept_id
- type
- title
- aliases
- tags
- path

schema_index
- global_id
- table
- column
- type
- description
- aliases

link_index
- source_global_id
- target_global_id
- relation
```

질의 시에는 `concept_index` → `schema_index` → 원문 `markdown` 순서로 좁혀가면 됩니다.

6. 최종 가이드

OKF를 LLM이 읽을 수 있는 **lightweight knowledge catalog**로 설계:

Directory Tree = 사람이 관리하는 구조
Markdown Links = concept 간 관계 graph
Frontmatter = agent discovery metadata
Generated Index = 빠른 schema/concept 검색용 catalog

1. Text-to-SQL agent에서 OKF의 역할

```
User Question
↓
OKF 검색
↓
관련 bundle 선택
↓
table/schema/metric/join concept 찾기
↓
SQL 생성용 context 구성
↓
SQL 생성
↓
실행/검증/응답
```

OKF spec에서 concept은 “하나의 markdown 문서”이고, table, API, metric, business process 같은 자산이나 개념을 표현할 수 있습니다. concept ID는 bundle 내부 파일 경로에서 나옵니다. 예를 들어 `tables/users.md`는 `tables/users` concept입니다. ([GitHub](#))

Text-to-SQL에서는 OKF concept을 이렇게 나누면 좋습니다.

<code>tables/customers.md</code>	→ 물리 테이블 설명
<code>schemas/customers.md</code>	→ 컬럼, 타입, PK/FK
<code>metrics/vip_customers.md</code>	→ 비즈니스 지표 정의
<code>joins/customer_orders.md</code>	→ 조인 경로
<code>examples/vip_query.md</code>	→ 검증된 SQL 예제
<code>glossary/vip.md</code>	→ “VIP”의 업무 의미

2. OKF는 SQL catalog + business semantics layer

DBMS의 `information_schema`가 주는 정보는 보통 이 정도입니다.

```
table name
column name
data type
primary key
foreign key
index
constraint
```

하지만 Text-to-SQL agent가 진짜로 필요한 것은 더 많습니다.

“VIP 고객”이 `customers.grade = 'VIP'`인지?
“매출”이 `orders.amount`인지 `payments.paid_amount`인지?
취소 주문을 제외해야 하는지?

회원 수는 distinct user_id인지 row count인지?
어떤 테이블끼리 조인해야 안전한지?

이런 정보는 DBMS schema만으로 부족합니다. OKF bundle은 이 부족한 부분을 채우는 **LLM-readable semantic catalog**입니다.

Google Cloud의 Knowledge Catalog 문서도 데이터베이스 metadata discovery가 table, column, view, PK/FK 같은 기술 메타데이터를 수집하고, aspect를 붙여 비즈니스/운영 메타데이터를 보강하는 방식이라고 설명합니다. ([Google Cloud Documentation](#))

3. OKF KB에 적절한 관계연산

OKF KB를 내부적으로는 다음과 같은 relation들로 정규화해서 다루면 좋습니다.

```
Concept(bundle_id, concept_id, type, title, path)
Table(concept_id, table_name, description)
Column(table_concept_id, column_name, data_type, meaning, aliases)
Metric(concept_id, name, formula, grain, filters)
JoinRule(source_table, target_table, join_condition, cardinality)
Alias(term, target_concept_id)
Example(question, sql, related_concepts)
Link(source_concept_id, target_concept_id, relation_type)
```

그 위에서 쓰는 관계연산은 다음이 핵심입니다.

관계연산	OKF에서의 의미
Selection σ	query와 관련 있는 concept만 고르기
Projection π	SQL 생성에 필요한 column/schema만 추출
Join $\bowtie$	table concept과 column, metric, join rule 연결
Semi-join $\ltimes$	후보 table에 속한 column만 남기기
Union $\cup$	여러 bundle 검색 결과 합치기
Difference $-$	권한 없는 schema, deprecated concept 제외
Group/Aggregate	table별 후보 점수, bundle별 relevance 계산
Recursive traversal	concept link를 따라 metric $\rightarrow$ table $\rightarrow$ column $\rightarrow$ join 탐색

예를 들어 “VIP 고객은 몇 명이야?”는 OKF KB에서 이렇게 처리됩니다.

```
 $\sigma$  aliases.term = "VIP"
   $\rightarrow$  glossary/vip

glossary/vip  $\bowtie$  related_concepts
   $\rightarrow$  tables/customers

tables/customers  $\bowtie$  columns
   $\rightarrow$  customers.grade, customers.id

metric rule 확인
   $\rightarrow$  count customers where grade = 'VIP'
```

그 결과 SQL은:

```
SELECT COUNT(*) AS vip_customer_count
FROM customers
```

```
WHERE grade = 'VIP';
```

4. Text-to-SQL agent 내부 pipeline

가장 실용적인 구조는 다음과입니다.

1. Bundle Routing
 - 이 질문이 ecommerce인지, GA4인지, StackOverflow인지 선택
2. Schema Linking
 - 관련 table, column, metric 후보 검색
3. Semantic Resolution
 - “VIP”, “활성 사용자”, “매출” 같은 업무 용어를 조건/공식으로 변환
4. Join Planning
 - 필요한 table 간 join path 선택
5. SQL Draft
 - SELECT/FROM/JOIN/WHERE/GROUP BY 생성
6. SQL Validation
 - 존재하지 않는 컬럼 제거
 - 허용되지 않은 table 차단
 - grain mismatch 검사
7. Execution & Answer

Text-to-SQL 연구에서도 schema linking은 SQL 생성 정확도에 큰 영향을 주는 핵심 단계로 다뤄집니다. 최근 연구는 먼저 table 후보를 좁히고, 그 안에서 column을 고르는 계층적 schema linking이 중요하다고 설명합니다. ([arXiv](#))

5. OKF bundle을 어떻게 이해해야 하나?

DBMS/SQL 세계에서는 다음이 핵심 concept입니다.

Database
Schema
Table
Column
Row
Primary Key
Foreign Key
View
Query
Join
Transaction

OKF bundle 세계에서는 대응되는 개념이 약간 다릅니다.

Bundle = 하나의 지식 패키지 / domain catalog
Concept = table, metric, glossary, rule, example 등을 담은 문서
Concept ID = bundle 내부 path 기반 ID
Frontmatter = machine-readable metadata
Markdown body = human/LLM-readable explanation
Link = concept 간 관계

즉:

DBMS schema = 실행 가능한 데이터 구조

OKF bundle = LLM이 그 데이터 구조를 올바르게 이해하게 하는 설명 구조

6. 추천 구현 방식

Text-to-SQL agent용 OKF bundle에는 최소한 이 6종 concept을 두는 것이 좋습니다.

```
tables/  
  customers.md  
  orders.md  
  
columns/  
  또는 table 문서 내부 schema section  
  
metrics/  
  revenue.md  
  vip_customer_count.md  
  
joins/  
  customer_orders.md  
  
glossary/  
  vip.md  
  active_user.md  
  
examples/  
  vip_customers.sql.md
```

그리고 agent는 SQL을 만들 때 반드시 다음 순서로 OKF를 읽게 해야 합니다.

```
glossary → metric → table → column → join → example
```

최종적으로 OKF는 **LLM용 data dictionary + semantic layer + verified query guide**입니다. DBMS가 “무엇을 실행할 수 있는가”를 정의한다면, OKF는 “LLM이 무엇을 어떻게 물어보고 해석해야 하는가”를 정의합니다.

올려주신 병원 예제를 보니, 기존 `techshop.db(customers, orders, products...)`에 병원 도메인(`doctors`, `patients`, `appointments`...)을 추가하려는 구조네요.

이 경우 제가 추천하는 것은 DB 구조를 그대로 **llm-wiki**에 복사하지 않는 것입니다.

DB는 **Physical Model**이고, OKF는 **Semantic Model**이어야 합니다.

제가 추천하는 project 구조

```
project-root/  
|  
├── data/  
|   ├── techshop.db  
|   └── seed/  
|  
├── llm-wiki/  
|   |  
|   ├── wiki.yaml          # 전체 catalog  
|   ├── glossary/  
|   |   vip.md  
|   |   revenue.md
```

```

|
|
|   |----- bundles/
|   |
|   |   techshop/
|   |   bundle.yaml
|   |   index.md
|   |
|   |   tables/
|   |   schemas/
|   |   joins/
|   |   metrics/
|   |   examples/
|   |
|   |   hospital/
|   |   bundle.yaml
|   |   index.md
|   |
|   |   tables/
|   |   schemas/
|   |   joins/
|   |   metrics/
|   |   glossary/
|   |   examples/
|   |
|   |----- shared/
|   |   sql_rules.md
|   |   prompt_templates.md
|
|----- src/

```

이렇게 하면 bundle 하나가 하나의 Domain이 됩니다.

Hospital bundle

예를 들면

```

hospital/
  tables/
    doctors.md
    patients.md
    appointments.md
    prescriptions.md
    insurance.md
  schemas/
    doctors_schema.md
    patients_schema.md
  joins/
    patient_doctor.md
    patient_insurance.md
    appointment_record.md
  metrics/
    doctor_count.md
    patient_count.md
    active_patient.md
    available_slots.md

```



```
glossary/
  specialty.md
  예약.md
  진료.md
examples/
  doctor_queries.md
  patient_queries.md
```

Table concept

예를 들어

```
tables/doctors.md
```

에는

```
---
type: table
table: doctors
primary_key: doctor_id
---

# Doctors

병원의 의사 정보를 저장한다.

Related

- patients
- appointments
- availability

Business Meaning

한 명의 의사는 여러 예약을 가진다.
```

여기에는 SQL은 거의 없습니다.

LLM이 이해하기 위한 설명입니다.

Schema concept

```
schemas/doctors_schema.md
```

```
---
type: schema
table: doctors
---

| column | type | meaning |
|-----|-----|-----|
doctor_id | varchar | 의사ID
name | varchar | 이름
specialty | varchar | 전공
hospital_name | varchar | 병원명
```

```
office | varchar | 진료실
...
```

여기는 SQL 생성을 위한 Metadata입니다.

Join concept

Text-to-SQL에서 가장 중요한 것이 Join입니다.

```
joins/patient_doctor.md
```

```
Patients
  |
patient_id

Appointments

doctor_id
  |

Doctors
```

그리고

```
Join Rule

patients.patient_id
    =
appointments.patient_id

appointments.doctor_id
    =
doctors.doctor_id
```

그리고

```
Typical Query

환자의 담당 의사

예약한 의사

의사의 예약목록
```

이런 설명이 들어갑니다.

LLM은 이 문서를 읽고 Join Path를 찾습니다.

Metric concept

Metric은 SQL보다 더 중요합니다.

예를 들면

```
metrics/available_slots.md
```

```
Definition

예약 가능한 시간
```

Formula

availability.status='가능'

Table

availability

Dimension

doctor

date

이런 Metric이 있어야

"김지훈 의사의 예약 가능한 시간"

이라는 질문을 SQL로 만들 수 있습니다.

Example concept

예를 들면

examples/doctor_queries.md

Question

김지훈 의사 이메일

SQL

```
SELECT email
FROM doctors
WHERE name='김지훈'
```

Question

의사 수

SQL

```
SELECT COUNT(*)
FROM doctors
```

이런 Few-shot Example입니다.

Bundle Manifest: bundle.yaml

```
bundle: hospital
database: techshop.db
tables:
  doctors
  patients
  appointments
  prescriptions
```

```
insurance
metrics:
  available_slots
  patient_count
entry_tables:
  patients
  doctors
  appointments
description:
  Hospital Management System
```

Text-to-SQL를 위한 "Schema Discovery"

```
Knowledge Catalog(Bundle) Selection
↓
Question
↓
Table Discovery
↓
Join Discovery
↓
Metric Discovery
↓
Example Retrieval
↓
SQL Generation
↓
SQL Validation
↓
Execution
```

추천 구조

OKF llm-wiki는 **Planner**가 사용할 **Knowledge Catalog**입니다.

- **SQLite**는 Physical Database
- **OKF Bundle**은 Semantic Database
- **Planner**는 Semantic Query Optimizer
- **SQL Generator**는 Code Generator

```
llm-wiki
↓
Planner Agent
↓
Logical Query Plan
↓
SQL Generator
↓
SQLite
```

이 구조는 **kchef Agent** → **Planner** → **Executor(Text-to-SQL)** → **Validator** 워크플로와도 매우 잘 맞습니다. Planner는 먼저 OKF에서 의미와 스키마를 찾아 논리적 질의 계획(Logical Query Plan)을 세우고, 그 계획을 기반으로 Executor가 실제 SQL을 생성·실행하도록 역할을 명확히 분리할 수 있습니다.